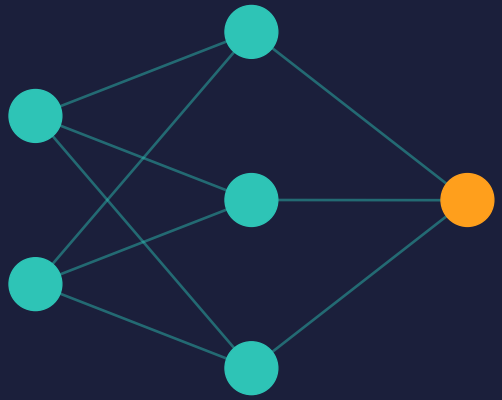
دوره یادگیری عمیق و شبکه‌های عصبی

جلسه ۱

از یادگیری ماشین تا یادگیری عمیق: Perceptron و MLP



نورون مصنوعی، مسئله XOR، توابع فعال‌سازی و شبکه چندلایه

کیوان محمودی

در این جلسه

۱ چرا یادگیری عمیق؟ Representation Learning

۲ نورون مصنوعی و Perceptron

۳ قانون یادگیری Perceptron

۴ مسئله XOR و محدودیت مدل خطی

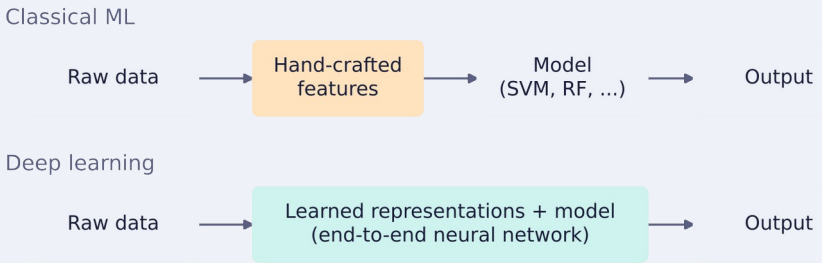
۵ لایه پنهان و MLP

۶ توابع فعال‌سازی (Activation Functions)

۷ Forward Pass و Softmax

۸ قضیه تقریب جهانی (Universal Approximation)

از Feature Engineering تا Representation Learning



- در ML کلاسیک کیفیت مدل به ویژگی‌های طراحی‌شده توسط انسان وابسته است.
- برای تصویر، متن و صوت طراحی دستی ویژگی سخت و پرهزینه است.
- شبکه عصبی از داده خام، نمایش‌های لایه‌به‌لایه را خودش یاد می‌گیرد.
- به این رویکرد End-to-End Learning می‌گوییم.

چرا همین حالا؟ سه عامل رشد یادگیری عمیق

۱ داده (Data)

دیتاست‌های بزرگ مثل ImageNet با بیش از یک میلیون تصویر برچسب‌دار و متن‌های عظیم وب.

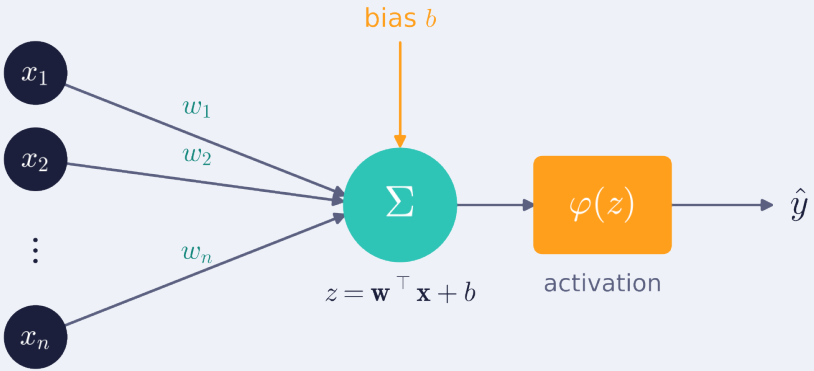
۲ سخت‌افزار (Compute)

GPUها ضرب ماتریسی را هزاران برابر موازی انجام می‌دهند؛ AlexNet در ۲۰۱۲ روی دو GPU آموزش دید.

۳ الگوریتم (Algorithms)

ReLU، Dropout، BatchNorm، ResNet، Adam و Transformer آموزش شبکه‌های بسیار عمیق را ممکن کردند.

نورون مصنوعی (Artificial Neuron)



- هر ورودی در یک وزن (Weight) ضرب می‌شود.
- جمع وزن‌دار به‌علاوه بایاس (Bias) مقدار z را می‌سازد.
- z از تابع فعال‌سازی عبور می‌کند.
- در Perceptron تابع فعال‌سازی پله‌ای (Step) است.
- مرز تصمیم یک ابرصفحه است؛ پس Perceptron یک طبقه‌بند خطی است.

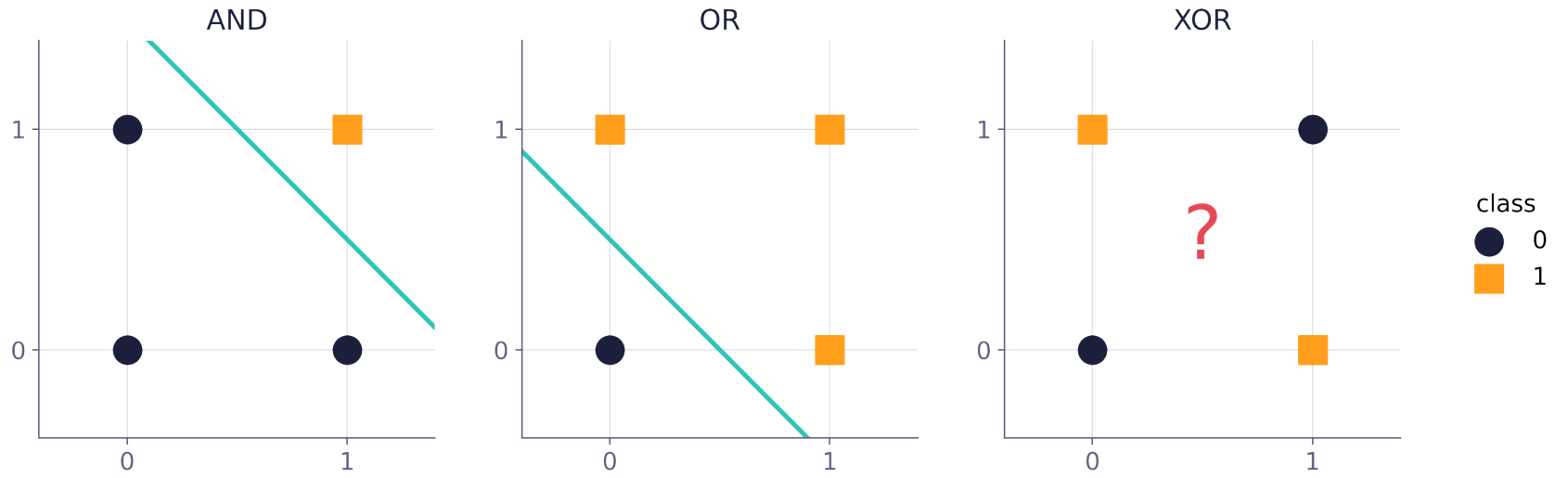
قانون یادگیری Perceptron

$$\mathbf{w} \leftarrow \mathbf{w} + \eta\,(y - \hat{y})\,\mathbf{x}$$

$$b \leftarrow b + \eta\,(y - \hat{y})$$

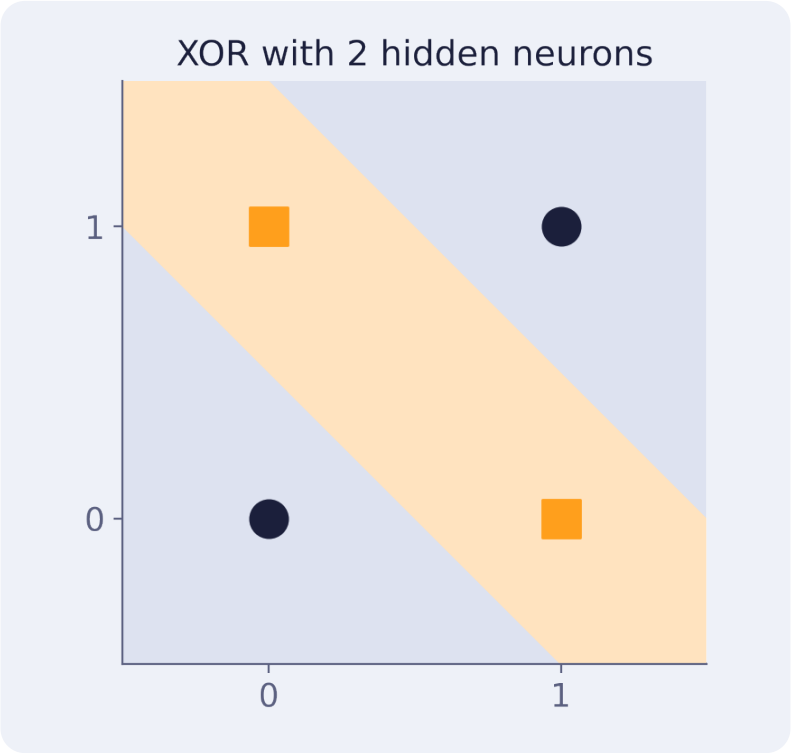
- اگر پیش‌بینی درست باشد ($y = \hat{y}$) وزن‌ها تغییر نمی‌کنند.
- اگر اشتباه باشد، وزن‌ها در جهت کاهش خطا جابه‌جا می‌شوند؛ η نرخ یادگیری (Learning Rate) است.
- برای داده خطی‌جداپذیر همگرایی در تعداد گام محدود تضمین شده است.

مسئله XOR: جایی که خط راست کافی نیست



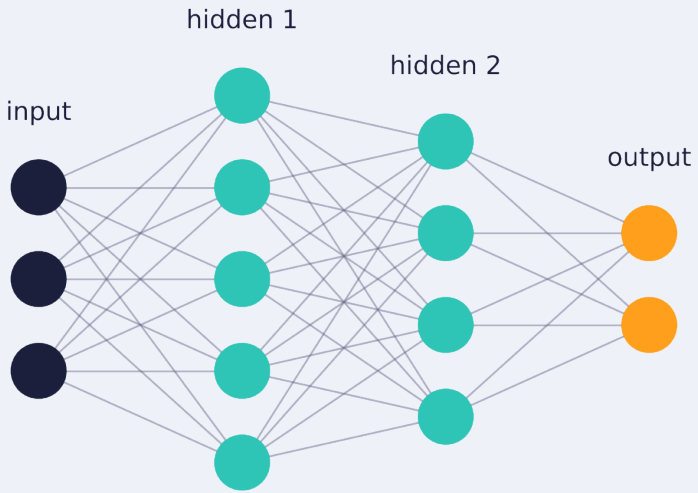
AND و OR با یک خط جدا می‌شوند، اما برای XOR هیچ خط راستی وجود ندارد (Minsky & Papert, 1969)

راه‌حل: لایه پنهان (Hidden Layer)



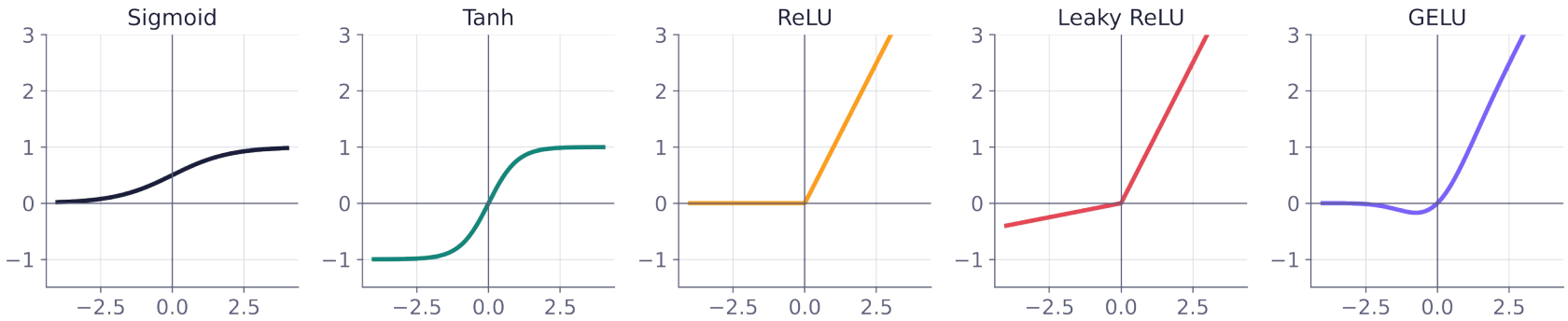
- $XOR = AND(OR(x_1,x_2) , NAND(x_1,x_2))$
- لایه پنهان با دو نورون OR و NAND داده را به فضای جدیدی می‌برد.
- در فضای جدید، مسئله خطی‌جداپذیر می‌شود و نورون خروجی آن را حل می‌کند.
- ایده اصلی یادگیری عمیق: هر لایه نمایش تازه‌ای از داده می‌سازد.

پرسپترون چندلایه (Multi-Layer Perceptron)



- لایه ورودی، یک یا چند لایه پنهان و لایه خروجی.
- هر نورون به همه نورون‌های لایه بعد وصل است: Fully Connected یا Dense.
- هر لایه یک ماتریس وزن W و یک بردار بایاس b دارد.
- تعداد پارامتر لایه: n_in × n_out + n_out

توابع فعال‌سازی رایج



ReLU پیش‌فرض لایه‌های پنهان است، GELU در Transformerها و Sigmoid/Softmax در لایه خروجی

چرا غیرخطی بودن ضروری است؟

بدون Activation

- ترکیب دو لایه خطی، باز هم یک لایه خطی است.
- $W_2(W_1x + b_1) + b_2 = Wx + b$
- عمق شبکه هیچ قدرت بیان اضافه‌ای ایجاد نمی‌کند.

با Activation غیرخطی

- هر لایه فضا را خم و تا می‌کند.
- شبکه می‌تواند مرزهای تصمیم پیچیده بسازد.
- مشتق‌پذیری لازم است تا با گرادیان آموزش ببیند (جلسه ۲).

Softmax و Forward Pass

$$H^{[l]} = \varphi\left(H^{[l-1]}\,W^{[l]} + \mathbf{b}^{[l]}\right), \qquad H^{[0]} = X$$

$$\mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

- ورودی به صورت Batch پردازش می‌شود: X با ابعاد (batch_size, n_features).
- خروجی‌های قبل از Softmax را Logits می‌نامیم.
- Softmax خروجی را به توزیع احتمال روی کلاس‌ها تبدیل می‌کند.

پیاده‌سازی MLP با NumPy

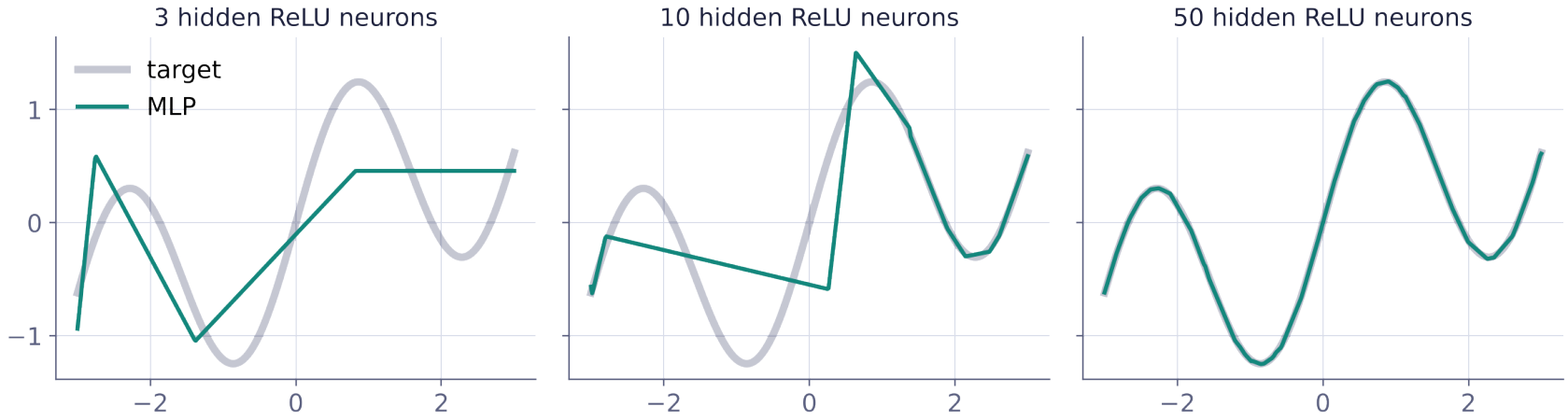
```
class MLP:
    def __init__(self, sizes, seed=0):
        rng = np.random.default_rng(seed)
        self.params = [
            (rng.normal(0, np.sqrt(2 / n_in), (n_in, n_out)),
             np.zeros(n_out))
            for n_in, n_out in zip(sizes[:-1], sizes[1:])]

    def forward(self, X):
        h = X
        for i, (W, b) in enumerate(self.params):
            z = h @ W + b
            last = i == len(self.params) - 1
            h = softmax(z) if last else relu(z)
        return h

mlp = MLP([4, 16, 8, 3])   # 243 parameters
```

- هر لایه یک ضرب ماتریسی است.
- ReLU در لایه‌های پنهان و Softmax در خروجی.
- وزن‌ها هنوز تصادفی‌اند؛ آموزش در جلسه بعد.

قضیه تقریب جهانی (Universal Approximation)



هر نورون ReLU یک شکستگی می‌سازد؛ با نورون‌های بیشتر، تقریب منحنی هدف دقیق‌تر می‌شود

جمع‌بندی

- نورون مصنوعی = جمع وزن‌دار + بایاس + تابع فعال‌سازی
- Perceptron طبقه‌بند خطی است و XOR را حل نمی‌کند.
- لایه پنهان نمایش جدیدی می‌سازد و مسائل غیرخطی را حل‌پذیر می‌کند.
- بدون Activation غیرخطی، عمق بی‌فایده است.
- MLP تقریب‌زننده جهانی است، ولی در عمل عمق کارآمدتر از پهنا است.

تمرین

۱ وزن‌های یک Perceptron را برای تابع NAND تعیین کنید.

۲ Perceptron را روی دو خوشه هم‌پوشان اجرا کنید و نمودار خطا در هر Epoch را تحلیل کنید.

۳ تعداد پارامترهای MLP با ساختار [784, 256, 128, 10] را حساب کنید.

۴ در آزمایش تقریب جهانی، تابع هدف را |x| و سپس پله‌ای کنید؛ کدام سخت‌تر است؟